

Path Planning for a Robot

Name : Asmit Sahu Roll : 23052231

```
In [ ]: import heapq
import math
import matplotlib.pyplot as plt
from collections import deque

# manhattan distance and euclidian dist
def manhattan(a, b):
    return abs(a[0]-b[0]) + abs(a[1]-b[1])

def euclidean(a, b):
    return math.sqrt((a[0]-b[0])**2 + (a[1]-b[1])**2)

# A* Search
def astar(grid, start, goal, allow_diagonal=False):

    rows, cols = len(grid), len(grid[0])

    if allow_diagonal:
        directions = [(1,0),(-1,0),(0,1),(0,-1),(1,1),(1,-1),(-1,1),(-1,-1)]
        heuristic = euclidean
    else:
        directions = [(1,0),(-1,0),(0,1),(0,-1)]
        heuristic = manhattan

    open_list = []
    heapq.heappush(open_list, (0, start))

    came_from = {}
    g_cost = {start: 0}

    while open_list:
        _, current = heapq.heappop(open_list)

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for dx, dy in directions:
            nx, ny = current[0] + dx, current[1] + dy

            if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == 0:
                new_cost = g_cost[current] + 1
```

```
    if (nx, ny) not in g_cost or new_cost < g_cost[(nx, ny)]:  
        g_cost[(nx, ny)] = new_cost  
        f_cost = new_cost + heuristic((nx, ny), goal)  
        heapq.heappush(open_list, (f_cost, (nx, ny)))  
        came_from[(nx, ny)] = current
```

```
    return None
```

```
In [ ]: grid = [  
    [0,0,0,0,0],  
    [1,1,0,1,0],  
    [0,0,0,1,0],  
    [0,1,1,0,0],  
    [0,0,0,0,0]  
]  
  
start = (0,0)  
goal = (4,4)  
  
path = astar(grid, start, goal, allow_diagonal=False)  
  
print("Path:", path)
```

```
Path: [(0, 0), (0, 1), (0, 2), (0, 3), (0, 4), (1, 4), (2, 4), (3, 4), (4, 4)]
```